

Execution, History, and Reduction: Toward a Minimal Operational Kernel of Computation

Flyxion

August 13, 2026

Abstract

Modern computational theory frequently treats programs as static objects whose meaning is determined by membership relations over sets of states. Yet real systems operate through irreversible event histories rather than through timeless configurations. This essay proposes and develops a minimal operational kernel in which event histories constitute the primary structure of computation, while states and abstractions emerge as derived reductions of those histories. Three primitive operations govern this kernel: extension, which appends events to histories and so realizes execution; merge, which reconciles compatible histories through a join-like operation; and reduction, which compresses histories into derived representations through controlled information loss. From the interaction of these operations arise deterministic replay, distributed convergence, log-structured computation, and constraint-driven dynamics. The algebra induced by these operations is shown to correspond to a join-semilattice structure over a partially ordered space of histories, admitting a monotone potential whose descent characterizes execution. The framework thereby replaces set-theoretic membership with replayable construction histories, providing a unified account of execution, composition, and abstraction applicable both to contemporary computational architectures and to physical systems governed by locally propagated constraints.

Contents

1	The Failure of State Ontology	5
1.1	The Snapshot Assumption and Its Discontents	5
1.2	The Erasure of Causal Structure	6
1.3	The Irreversibility of Execution	6
1.4	Toward an Event-Historical Ontology	7

2	Event Histories as the Primitive Structure	7
2.1	Events	7
2.2	Histories	8
2.3	Partial Ordering of Events	8
2.4	Derived State	9
3	The Minimal Operational Kernel	9
3.1	Extension	9
3.2	Merge	10
3.3	Reduction	11
3.4	Kernel Sufficiency	11
4	Determinism and Replayability	12
4.1	Replay and Determinism	12
4.2	State as Cache	12
4.3	Causal Correctness	13
5	Composition Through History Merge	13
5.1	Divergent Branches and Reconciliation	13
5.2	Causal Preservation as a Correctness Condition	14
5.3	Conflict and Compensation	14
5.4	Examples from Practice	14
6	Abstraction as Reduction	15
6.1	Reduction as Information Loss	15
6.2	Layered Reductions	15
6.3	Abstraction and Algorithmic Complexity	16
7	The Algebra of Histories	17
7.1	The History Poset	17
7.2	Monotonicity of Extension	17
7.3	Merge as Join	17
7.4	Deterministic Replay	18
7.5	Reduction and Information Loss	18
7.6	The Monotone Potential	19
8	Time, Causality, and Irreversibility	19
8.1	Temporal Structure from Causal Order	19

8.2	Two Modes of Irreversibility	20
8.3	Event Histories and Constraint Propagation in Physical Systems	20
8.4	Monotone Constraint Dynamics	21
9	Unified Constraint-Driven Computation	22
9.1	Deterministic Replay Systems	22
9.2	Distributed Convergence	22
9.3	Version-Control Systems	23
9.4	Constraint-Based Programming	23
9.5	Common Algebraic Structure	23
10	Conclusion	24

Introduction

The dominant traditions of theoretical computer science describe programs in terms of transformations between states. From early automata theory to modern programming-language semantics, computational systems are modeled as structures that occupy well-defined configurations and evolve by transitioning among them. These models have proven extraordinarily productive for reasoning about correctness, complexity, and program behavior, furnishing the foundations of the discipline.

Yet the systems that computing actually relies upon increasingly strain the assumptions underlying state-centered models. Large-scale distributed services, append-only logs, version-control systems, and event-sourced architectures do not treat state as the primary object of computation. They record sequences of events whose cumulative effects determine the observable condition of the system. In such architectures, the present configuration is best understood not as a self-contained object but as a compressed summary of the history that produced it. The ordering of events, the causal dependencies among them, and the constraints propagated through their interactions carry information that cannot be recovered from a snapshot taken at any single moment.

This observation suggests a different foundational starting point. Rather than treating states as primitive and histories as auxiliary traces or mere byproducts of execution, one may reverse the relationship and regard histories as the fundamental structures from which states are derived. Such a reversal is not merely terminological; it reshapes the conceptual apparatus available for analyzing execution, composition, and abstraction.

The present essay develops a minimal operational framework built upon this reversal. In this framework, computation is interpreted as the irreversible construction of event histories under locally propagated constraints. System states emerge as reductions of those histories, and abstraction itself is understood as a form of controlled information compression applied to execution traces. The resulting kernel is minimal in the sense that three operations—extension, merge, and reduction—suffice to generate the full space of computationally relevant structures, and no proper subset of these operations is adequate to this task.

The argument proceeds as follows. Section 1 examines the structural limitations of state-centered ontologies of computation and motivates the transition to an event-historical perspective. Section 2 introduces event histories as the primitive objects of the theory and derives states as projections of those histories. Section 3 defines the minimal operational kernel and characterizes each of its three operations. Sections 4 through 6 develop the operational consequences of the kernel for determinism, composition, and abstraction respectively. Section 7 formalizes the algebraic structure induced by the kernel operations, establishes its principal properties, and proves the main structural theorems. Section 8 investigates the emergence of temporal structure, causality, and irreversibility from within the framework. Section 9 traces the common algebraic pattern across a range of computational

architectures and physical dynamical systems. Section 10 draws together the conclusions of the analysis.

1 The Failure of State Ontology

The choice of a foundational ontology for the theory of computation is not a purely formal matter. It conditions what can be expressed within the theory, what questions the theory is capable of posing, and what phenomena are visible within its conceptual framework. The state-centered ontology that has dominated computational theory since its inception carries specific assumptions that, while natural from one perspective, prove limiting when applied to the class of systems that contemporary computation relies upon.

1.1 The Snapshot Assumption and Its Discontents

The core commitment of state-based computation is what we may call the snapshot assumption: the thesis that the complete condition of a system at any moment can be captured by a snapshot—a self-contained, instantaneously observable configuration. Formal automata, transition systems, and denotational semantics all share this commitment. The state of a system is taken to be a well-defined element of some set S , and computation is modeled as a sequence of transitions

$$s_0 \xrightarrow{\tau_1} s_1 \xrightarrow{\tau_2} s_2 \xrightarrow{\tau_3} \dots$$

in which each state s_i fully encodes the condition of the system at time i .

This picture is well-suited to sequential, isolated, and reversible computation. It captures finite automata, Turing machines, and the denotational semantics of pure functional programs with considerable clarity. The theoretical yield of this framework—decidability results, complexity classifications, program logics, type theories—has been immense.

What the snapshot assumption cannot easily accommodate, however, is the class of systems in which the present configuration is intelligible only in relation to the sequence of events that produced it. Distributed databases require that the history of transactions be preserved in order to maintain consistency guarantees across replicas. Version-control systems derive their entire meaning from the directed graph of commits that encodes the development of an artifact over time. Event-sourced architectures store operations rather than states precisely because the sequence of operations carries information that the resulting state alone does not. In each of these cases, a snapshot may summarize the system, but it does not explain how the system came to be, nor does it preserve the information required to verify that its current configuration is legitimate.

1.2 The Erasure of Causal Structure

When computation is modeled purely as state transformation, the causal structure of execution is systematically erased. Two systems may arrive at observationally indistinguishable states through radically different histories. These distinct histories can have implications for correctness, safety, auditability, and reproducibility that are completely invisible to any analysis conducted at the level of states alone.

The state of a distributed ledger, for instance, cannot be validated without examining the history of transactions that produced it. The integrity of a filesystem cannot be guaranteed by inspecting a snapshot if the sequence of write operations is unavailable. The meaning of a version-controlled repository is not located in the current working tree but in the commit graph that generated it. In each of these domains, history is not auxiliary metadata appended to a primary state; it is the primary structure of the system, and the state is its derived projection.

This erasure of causal structure is not a technical limitation of particular formalisms; it follows from the snapshot assumption itself. If a state is by hypothesis a complete description of the system, then no information that cannot be reconstructed from the state is, by definition, part of the system's condition. The causal history of how that state was reached is thereby classified as external and irrelevant. This classification is appropriate for many purposes, but it becomes a structural distortion when applied to systems in which historical ordering carries semantic weight.

1.3 The Irreversibility of Execution

Execution is fundamentally an irreversible process. Once an event has occurred—a message has been delivered, a transaction committed, a sensor reading recorded—that event cannot be undone without introducing additional compensating events that themselves become part of the system's history. This irreversibility is not a contingent feature of particular implementations; it reflects the temporal asymmetry of physical processes and of information-theoretic change more generally.

The state-transition model attempts to compress this irreversible process into a sequence of snapshots, treating each state as fully determined by its predecessor under the transition function. This compression is valid when transitions are deterministic and the snapshot captures all information relevant to future behavior. It becomes problematic in precisely those cases where the ordering of transitions, the concurrency among them, or the causal dependencies between them bear on the behavior of the system in ways that cannot be recovered from the current state.

The monotonic growth of execution—the fact that computation accumulates events rather than traversing a fixed state space—suggests that the most natural description of a computational system is not a trajectory through states but a growing structure of causally ordered events. This shift in perspective is the point of departure for the framework developed in the remainder of this essay.

1.4 Toward an Event-Historical Ontology

The analysis above motivates a reorientation of the foundational ontology. Rather than treating state as primitive and history as derived, we propose to treat history as primitive and state as derived.

Principle 1 (Event-Historical Primacy). *The primary structure of a computational system is the irreversible history of events through which that system has been constructed. System states are projections of this history rather than primitive objects of the theory.*

This shift is conceptually significant. It changes the unit of analysis from a configuration to a process of construction. It makes explicit what state-centered models obscure: that computation is a temporal, irreversible, and causally structured activity, not a timeless membership relation between configurations and transition functions. It also provides a natural framework for reasoning about distributed systems, version-controlled artifacts, event-sourced architectures, and other domains in which history is not merely a record of past states but the constitutive structure of the system itself.

The following section makes this ontology precise by introducing the formal objects that serve as primitives in the event-historical kernel.

2 Event Histories as the Primitive Structure

We develop here the primitive structures of the event-historical framework, beginning with the notion of an event as an atomic unit of execution and proceeding to the definition of histories, their partial-order structure, and the derivation of states as projections.

2.1 Events

The atom of computation in this framework is not a state or a configuration but an event: a record of an irreversible transition that has occurred within the system.

Definition 1 (Event). *An event is an irreversible atomic transition in a computational system. Formally, events are elements of a set $\mathcal{E}$, called the event universe, whose elements represent the class of observable transitions available to the system under consideration.*

Events are intentionally abstract. Their internal structure—whether they encode messages, computations, sensor readings, or state mutations—is not fixed by the theory. What matters for the algebraic development that follows is not the content of events but their ordering relations and their effects on the causal structure of the histories they inhabit. The choice to abstract away from event content is deliberate: it yields a framework applicable across diverse computational and physical domains.

2.2 Histories

A history is a causally ordered accumulation of events. The essential properties of a history are its monotonicity—events once appended cannot be removed—and its causal structure—the ordering among events records the dependencies that governed their occurrence.

Definition 2 (Event History). *An event history H is a finite or countably infinite sequence of events*

$$H = (e_1, e_2, e_3, \dots)$$

such that each event e_{k+1} represents a valid extension of the history $(e_1, \dots, e_k)$ determined by the system's transition rules.

The qualifier “valid extension” is crucial. Not every event may be appended to every history; the admissibility of an extension is governed by the causal and semantic constraints of the system. A transaction may not be committed before it has been initiated; a message may not be delivered before it has been sent. The validity condition encodes these dependencies and thereby ensures that histories respect causal order.

2.3 Partial Ordering of Events

In many systems events are not totally ordered. Independent operations may proceed concurrently, and the resulting histories may exhibit a more complex causal structure than a simple linear sequence. To accommodate this generality, we introduce a partial-order structure over events.

Definition 3 (Causal Order). *An event history may equivalently be represented as a pair $(H, \leq_H)$ in which H is a set of events and $\leq_H$ is a partial order over H such that $e_i \leq_H e_j$ if and only if e_i causally precedes e_j in the history.*

This definition generalizes the linear sequence: a totally ordered history is the special case in which $\leq_H$ is a total order. When events are concurrent—that is, when neither $e_i \leq_H e_j$ nor $e_j \leq_H e_i$ holds—they may, in principle, occur in either order without violating causal constraints. The partial-order representation captures this independence explicitly.

Remark 1. *The partial-order view of histories is closely related to Lamport's formalization of causality in distributed systems [11], wherein the causal precedence relation over events defines a consistent global ordering. The event-historical kernel generalizes this framework by making the causal structure of histories the primary object of analysis rather than a derived property of a state-based system.*

2.4 Derived State

With histories established as the primary objects, states may now be introduced as derived projections.

Definition 4 (State Projection). *A state projection is a deterministic function*

$$\sigma : \mathcal{H} \rightarrow S$$

that maps each admissible history to an element of a state space S . The value $\sigma(H)$ is called the state induced by history H .

Different projections may derive different aspects of the same history. The current contents of a memory region, the configuration of a filesystem, the balance of a transaction ledger, and the aggregate metrics reported by a monitoring system are all, in this framework, distinct projections applied to the same underlying event history. None of them is the state of the system in any absolute sense; each is a compressed representation of the history, retaining only the distinctions relevant to its particular purpose.

This observation does not diminish the practical importance of state representations. It relocates their theoretical status: they are derived tools for reasoning about systems rather than the foundational objects of the theory.

3 The Minimal Operational Kernel

Having established event histories as the primitive objects of the theory, we now identify the minimal set of operations sufficient to construct and transform those histories. The goal is parsimony: to characterize a set of operations that is jointly sufficient to generate the full range of computationally relevant structures while admitting no redundant member.

Three operations constitute the kernel. The first is *extension*, which appends a new event to an existing history and thereby realizes the execution of a transition. The second is *merge*, which reconciles two compatible histories into a common continuation, thereby realizing composition. The third is *reduction*, which maps a history to a compressed representation by discarding structural information, thereby realizing abstraction. We characterize each in turn.

3.1 Extension

Extension is the fundamental operation of execution. It takes an existing history and an event satisfying the system's admissibility conditions and produces a new history strictly containing the original.

Definition 5 (History Extension). *Let $H \in \mathcal{H}$ be an admissible history and let $e \in \mathcal{E}$ be an event such that appending e to H preserves the causal ordering and satisfies the system's validity conditions. The extension operator is then a partial function*

$$\text{ext} : \mathcal{H} \times \mathcal{E} \rightarrow \mathcal{H}$$

defined by $\text{ext}(H, e) = H \cdot e$, where $H \cdot e$ denotes the history obtained by appending e to H as a maximal element under the causal order.

The partiality of ext is essential: not every event may be appended to every history. The domain of ext is exactly the set of pairs (H, e) for which the extension is causally admissible. Outside this domain, the operation is undefined.

Principle 2 (Monotonic Growth). *For any admissible pair (H, e) , the extended history satisfies $H \preceq H \cdot e$ under the prefix ordering. Equivalently, extension is a monotone operation with respect to causal containment.*

Monotonic growth formalizes the irreversibility of execution: no extension of a history can remove events already recorded within it.

3.2 Merge

Merge realizes composition. In distributed or concurrent systems, independent processes extend their local histories simultaneously. When their contributions are to be combined—for instance, when two replicas synchronize, two branches are reconciled, or two concurrent processes are joined—the resulting joint history must incorporate the events of both predecessors while preserving their respective causal orderings.

Definition 6 (History Merge). *Let $H_1, H_2 \in \mathcal{H}$ be histories that share a common prefix $H_0 \preceq H_1$ and $H_0 \preceq H_2$. The histories are compatible if their events can be combined without violating causal constraints or system invariants. For compatible H_1 and H_2 , the merge operation produces a history*

$$H_1 \sqcup H_2 \in \mathcal{H}$$

satisfying $H_1 \preceq H_1 \sqcup H_2$ and $H_2 \preceq H_1 \sqcup H_2$, and minimal with this property in the sense that for any H with $H_1 \preceq H$ and $H_2 \preceq H$, one has $H_1 \sqcup H_2 \preceq H$.

The minimality condition ensures that the merged history adds only what is necessary to subsume both branches. It corresponds precisely to the least upper bound, or join, in a partially ordered set,

and it is this correspondence that gives the space of histories its semilattice structure, as established in Section 7.

When histories are not directly compatible—because their events introduce contradictory constraints—merge requires the introduction of compensating events that restore consistency. These compensating events are themselves recorded in the merged history, preserving the irreversibility of execution while explicitly encoding the reconciliation process. Conflict resolution is therefore not erasure; it is history extension by another means.

3.3 Reduction

Reduction is the operation by which abstraction, summarization, and state derivation are realized within the framework. It maps a history to a compressed representation that preserves selected structural properties while discarding others.

Definition 7 (History Reduction). *A reduction operator is a function*

$$\rho : \mathcal{H} \rightarrow R$$

where R is a space of reduced representations. The operator ρ preserves a property P of histories if whenever H possesses P , $\rho(H)$ possesses the corresponding property in R .

Reductions are in general neither injective nor surjective. They are not injective because multiple distinct histories may compress to the same representation—this is precisely the information loss that characterizes abstraction. They need not be surjective because the space of valid representations may be smaller than the range of ρ can reach.

The most familiar reduction is the state projection $\sigma : \mathcal{H} \rightarrow S$ introduced in Section 2. But reductions include also the construction of audit logs that retain only security-relevant events, the generation of monitoring dashboards that aggregate performance metrics, the formation of compiler intermediate representations that retain only control-flow structure, and any other operation that derives a simplified description from a full execution trace.

3.4 Kernel Sufficiency

The three operations of extension, merge, and reduction are jointly sufficient to generate the space of computational structures considered in this framework. Extension constructs histories through execution. Merge composes histories through reconciliation. Reduction derives representations through compression. The interaction of these three operations generates deterministic replay, distributed consensus, version-controlled evolution, log-structured storage, and constraint-driven dynamics. The remainder of the essay develops the consequences of this kernel.

4 Determinism and Replayability

Within the event-historical framework, the concept of determinism must be reformulated. The classical notion—that a deterministic system always produces the same output given the same input, or equivalently that the transition function is a proper function rather than a relation—is stated at the level of states and transitions. Transposed into the language of histories, determinism becomes a property of replay rather than of transition.

4.1 Replay and Determinism

Definition 8 (Replay). *Given a history $H = (e_1, e_2, \dots, e_n)$ and a reduction operator $\sigma : \mathcal{H} \rightarrow S$, the replay of H under σ is the sequential application of the events $(e_1, \dots, e_n)$ to an initial configuration, producing the derived state $\sigma(H) \in S$.*

Definition 9 (Replay Determinism). *A system with reduction operator σ is replay-deterministic if $\sigma(H)$ is uniquely determined by H ; that is, if the mapping $\sigma : \mathcal{H} \rightarrow S$ is a well-defined function rather than a relation.*

Replay determinism is a weaker and more operationally grounded condition than classical state determinism. It does not require that the system produce identical event sequences when run repeatedly under identical conditions; non-determinism in execution may produce different histories. What it requires is that any given history, when replayed, produces a unique derived state. This is the condition that makes a history an authoritative record of execution.

4.2 State as Cache

Within the event-historical framework, the state of a system is best understood not as a primary object but as a cache derived from history. A snapshot represents the value of $\sigma(H)$ at some moment, stored to avoid the computational cost of replaying the full history H from scratch. The snapshot accelerates access to derived information but does not replace or supersede the history.

This interpretation has significant implications for system design and for the analysis of system failures. If a cached state is lost—through a crash, a hardware fault, or a software error—it can be reconstructed by replaying the underlying history. The history is therefore the authoritative source from which derived structures are rebuilt; the cache is a performance optimization rather than a primary datum. This architecture underlies distributed logging systems, database write-ahead logs, and event-sourced applications [7, 10], all of which maintain the event history as the durable, authoritative record.

4.3 Causal Correctness

The replay model induces a precise notion of correctness that is unavailable in purely state-based frameworks.

Principle 3 (Causal Correctness). *A derived state $s \in S$ is causally correct if there exists an admissible history $H \in \mathcal{H}$ such that $\sigma(H) = s$. A state that cannot be produced by replaying any admissible history is causally incorrect regardless of its syntactic validity.*

Causal correctness cannot be verified from the state alone; it requires access to the history from which the state was derived. This observation is especially significant in domains such as distributed ledgers, cryptographic audit logs, and fault-tolerant replicated state machines, where the authenticity of a configuration depends on its provenance from a sequence of verified events rather than on any intrinsic property of the configuration itself.

5 Composition Through History Merge

In classical programming models, composition is understood as a structural operation: programs are combined by placing them in sequential, parallel, or hierarchical arrangements. The state of the composed system is then some function of the states of the components. Within the event-historical framework, composition is instead understood as a historical operation: programs compose when their execution histories are merged into a common continuation.

5.1 Divergent Branches and Reconciliation

Execution frequently gives rise to divergent histories. Let H_0 be a common prefix, and suppose that two independent lines of execution extend it:

$$H_1 = H_0 \cdot A, \quad H_2 = H_0 \cdot B,$$

where $A = (a_1, \dots, a_m)$ and $B = (b_1, \dots, b_n)$ are sequences of events extending H_0 along independent causal paths. These histories represent two branches of execution whose events are, in the relevant sense, concurrent: neither branch's events causally precede the other's.

Composition arises when these branches are reconciled. The merged history

$$H_3 = H_1 \sqcup H_2$$

is a joint continuation that preserves the causal ordering within each branch while extending the common prefix H_0 . The events of A and B are incorporated into H_3 with their internal ordering

relations intact; only the ordering between events of A and events of B is left partially determined by their concurrent relationship.

5.2 Causal Preservation as a Correctness Condition

For a merge to be valid, it must preserve the causal ordering within each contributing history. This is not merely a technical requirement but a semantic one: causal ordering encodes the dependency structure of execution, and any merge that violated it would produce a history that misrepresented the actual sequence of events and could not be faithfully replayed.

Principle 4 (Causal Preservation under Merge). *For any events e_i, e_j with $e_i \leq_{H_1} e_j$ in H_1 , the merged history $H_1 \sqcup H_2$ satisfies $e_i \leq_{H_1 \sqcup H_2} e_j$. The same condition holds for the causal ordering within H_2 .*

Causal preservation ensures that merged histories remain replayable and that the provenance of each event is recoverable from the structure of the merged history.

5.3 Conflict and Compensation

When two histories introduce mutually inconsistent events—for instance, when two branches of a distributed transaction commit conflicting writes—no direct merge is defined. Consistency must instead be restored through compensating events: additional events that reverse or supersede the conflicting operations while recording the resolution explicitly in the extended history. The principle of monotonic growth ensures that the conflict itself is never erased from the historical record; it is only superseded by the compensating extension.

This treatment of conflict distinguishes the event-historical approach from models that resolve inconsistency by selecting one branch and discarding the other. In the event-historical framework, the conflicting history and its resolution are both preserved, providing a complete and auditable record of the system’s evolution.

5.4 Examples from Practice

The form of composition described here appears in a range of contemporary computational systems. Distributed version-control systems such as Git organize repository evolution as a directed acyclic graph of commits, in which each merge commit incorporates the events of two or more branches while preserving their internal ordering. Conflict-free replicated data types [17] achieve eventual consistency across distributed replicas precisely because their update operations form a join-semilattice: replicas that maintain divergent histories converge to a common state when their

histories are merged, without requiring centralized coordination. Event-sourced architectures [7] record all operations as append-only log entries and derive application state by replaying these logs, treating composition of services as the composition of their respective event streams. In each of these domains, the operational pattern is the same: independent histories are produced by parallel activity and subsequently reconciled through merge operations that preserve causal structure.

6 Abstraction as Reduction

The classical account of abstraction in programming theory is largely generative: new conceptual entities—types, modules, interfaces, abstract data types—are introduced to name and encapsulate collections of operations. Abstraction in this sense adds structure to the theory. Within the event-historical framework, by contrast, abstraction is understood as a subtractive operation: it removes distinctions from a history while preserving those relevant to a given purpose.

6.1 Reduction as Information Loss

The central mathematical property of reduction is non-injectivity.

Principle 5 (Irreversibility of Reduction). *For a reduction operator $\rho : \mathcal{H} \rightarrow R$, it is in general the case that*

$$\rho(H_1) = \rho(H_2) \not\Rightarrow H_1 = H_2.$$

Distinct histories may collapse to the same reduced representation under ρ .

This non-injectivity is not a deficiency of particular reduction operators; it is the defining characteristic of abstraction. The purpose of abstraction is precisely to identify a collection of distinct concrete situations with a single abstract description. The information discarded in doing so is the information that the abstraction deems irrelevant for the purposes at hand.

Once a history has been reduced, the information discarded by the reduction cannot in general be recovered from the reduced representation alone. Recovery requires either retaining the original history—which is the event-historical approach—or restricting the class of admissible histories so that the reduction becomes injective on that class.

6.2 Layered Reductions

Multiple distinct reduction operators may be applied to the same underlying history, yielding different representations of the same execution trace. A single operational history within a computational environment may simultaneously support a filesystem snapshot, a database view, a performance

monitoring summary, a security audit log, and a billing record. Each of these representations is derived from the same history by a distinct reduction that selects and preserves different aspects of the execution trace.

These derived structures should not be understood as independent realities. They are different informational projections of a single evolving history. Their apparent multiplicity reflects the existence of multiple purposes—storage management, data querying, performance optimization, security assurance, cost accounting—each of which defines a different reduction over the same underlying events. Layered abstractions are therefore not a fragmentation of the system’s state into independent components but a structured family of compressions applied to a common substrate.

6.3 Abstraction and Algorithmic Complexity

The relationship between abstraction and information loss can be made precise through the lens of algorithmic complexity. For a finite history H , define its *Kolmogorov complexity* as

$$K(H) = \min\{|p| : U(p) = H\},$$

where U is a fixed universal Turing machine and $|p|$ denotes the length of program p in some standard encoding. The quantity $K(H)$ measures the minimal description length from which the history can be reconstructed.

A reduction operator $\rho : \mathcal{H} \rightarrow R$ can be interpreted as a constrained compression: it maps H to a description $\rho(H)$ that is shorter than H but preserves selected observables. The description length of $\rho(H)$ is bounded below by $K(\rho(H))$, which may be substantially smaller than $K(H)$ if the reduction discards compressible structure. Unlike the Kolmogorov-optimal compression, however, a reduction is designed to preserve specific properties rather than to minimize description length globally. It is therefore a *semantically constrained compression* rather than an information-theoretically optimal one.

The set of histories consistent with a given reduced representation measures the information loss of the reduction. Define

$$\Omega(\rho, r) = |\{H \in \mathcal{H} : \rho(H) = r\}|$$

for a reduced state $r \in R$. The quantity $\log \Omega(\rho, r)$ is an entropy-like measure of the indeterminacy of the history given the reduced representation r . This quantity grows as the reduction discards more information, and it provides a precise sense in which more aggressive compression produces more fundamental irreversibility.

7 The Algebra of Histories

The three operations of the minimal kernel induce a natural algebraic structure over the space of event histories. We develop this structure here, establishing its principal properties and proving the theorems that govern the framework's behavior.

7.1 The History Poset

Let $\mathcal{H}$ denote the set of all admissible event histories generated by a fixed computational system.

Definition 10 (Prefix Order). *The prefix order on $\mathcal{H}$ is defined by*

$$H_1 \preceq H_2 \iff H_1 \text{ is a prefix of } H_2,$$

that is, H_1 is a causally embedded initial segment of H_2 under the causal ordering of H_2 .

Proposition 1. *The pair $(\mathcal{H}, \preceq)$ is a partially ordered set.*

Proof. Reflexivity holds because every history is a prefix of itself. Antisymmetry holds because if $H_1 \preceq H_2$ and $H_2 \preceq H_1$, then H_1 and H_2 are mutually initial segments of each other, which forces $H_1 = H_2$. Transitivity holds because if H_1 is a prefix of H_2 and H_2 is a prefix of H_3 , then H_1 is a prefix of H_3 . $\square$

7.2 Monotonicity of Extension

Theorem 1 (Monotonic Extension). *For any history $H \in \mathcal{H}$ and any event $e \in \mathcal{E}$ such that $\text{ext}(H, e)$ is defined,*

$$H \preceq \text{ext}(H, e).$$

Proof. By definition, $\text{ext}(H, e) = H \cdot e$ is the history obtained by appending e to H as a maximal element under the causal order. The events of H constitute an initial segment of $H \cdot e$ under this order. Hence H is a prefix of $H \cdot e$, which by definition gives $H \preceq H \cdot e$. $\square$

Corollary 1. *The extension operator is strictly monotone on non-trivially extended histories: if $\text{ext}(H, e)$ is defined, then $H \prec \text{ext}(H, e)$.*

7.3 Merge as Join

Theorem 2 (Merge Convergence). *Let $H_1, H_2 \in \mathcal{H}$ be compatible histories. If the merge $H_1 \sqcup H_2$ exists in $\mathcal{H}$, then it is the least upper bound of $\{H_1, H_2\}$ under the prefix order.*

Proof. We verify the three conditions for a least upper bound. First, by definition of merge, $H_1 \preceq H_1 \sqcup H_2$ and $H_2 \preceq H_1 \sqcup H_2$, so $H_1 \sqcup H_2$ is an upper bound. Second, suppose $H \in \mathcal{H}$ satisfies $H_1 \preceq H$ and $H_2 \preceq H$. Then H contains all events of H_1 and all events of H_2 as substructures, respecting the causal ordering of each. By the minimality condition in the definition of merge, $H_1 \sqcup H_2$ is the smallest history with this property, hence $H_1 \sqcup H_2 \preceq H$. Since H was arbitrary among upper bounds, $H_1 \sqcup H_2$ is the least upper bound. $\square$

Proposition 2. *When merge is defined for all compatible pairs in $\mathcal{H}$ and the compatibility relation is closed under iteration, the space $(\mathcal{H}, \preceq, \sqcup)$ forms a join-semilattice on the compatible subsets of $\mathcal{H}$.*

Proof. We must verify that the join operation is associative, commutative, and idempotent on compatible histories. Commutativity follows from the symmetric treatment of H_1 and H_2 in the definition of $\sqcup$: the least upper bound of $\{H_1, H_2\}$ is the same as the least upper bound of $\{H_2, H_1\}$. Idempotency follows because the least upper bound of $\{H, H\}$ is H itself. Associativity follows because the least upper bound of $\{H_1 \sqcup H_2, H_3\}$ is the least history containing H_1 , H_2 , and H_3 , which equals the least upper bound of $\{H_1, H_2 \sqcup H_3\}$. $\square$

7.4 Deterministic Replay

Theorem 3 (Deterministic Replay). *Let $\sigma : \mathcal{H} \rightarrow S$ be a reduction operator and suppose the semantics of each event are deterministic. Then $\sigma(H)$ is uniquely determined by H ; that is, σ is a well-defined function.*

Proof. Replay constructs $\sigma(H)$ by applying the events of H in their causal order to an initial configuration. Since the causal order of H is a partial order, and since replay respects this order by processing events in a causal linearization, any two linearizations of H that are consistent with its causal order produce the same final configuration when the event semantics are deterministic. This follows from the Church–Rosser property of deterministic rewriting systems: if the order of application does not affect the result of each deterministic step, then all causal linearizations produce the same outcome. Hence $\sigma(H)$ is independent of the linearization chosen and depends only on H . $\square$

7.5 Reduction and Information Loss

Proposition 3 (Non-Injectivity of Reduction). *For any non-trivial reduction operator $\rho : \mathcal{H} \rightarrow R$, there exist histories $H_1, H_2 \in \mathcal{H}$ with $H_1 \neq H_2$ and $\rho(H_1) = \rho(H_2)$.*

Proof. A trivial reduction would be the identity on $\mathcal{H}$, which retains all information and maps no distinct histories to the same representation. Any non-trivial reduction ρ discards at least one bit of distinguishing information between histories; hence there exist distinct histories that it maps to the same element of R . Formally, if ρ were injective, then $|R| \geq |\mathcal{H}|$, meaning ρ retains enough information to reconstruct any history from its image. But this contradicts the assumption that ρ compresses: a compression that admits a unique inverse is not a compression in the relevant sense but a recoding. $\square$

7.6 The Monotone Potential

To characterize execution dynamics within the history algebra, we introduce a potential function that measures the degree to which a history satisfies the system's constraints.

Definition 11 (Monotone Potential). *A monotone potential is a function $E : \mathcal{H} \rightarrow \mathbb{R}$ such that*

$$H_1 \preceq H_2 \Rightarrow E(H_2) \leq E(H_1).$$

Execution therefore proceeds in the direction of non-increasing potential.

Definition 12 (Stable History). *A history $H \in \mathcal{H}$ is stable if no valid extension $H' = \text{ext}(H, e)$ satisfies $E(H') < E(H)$. Stable histories are fixed points of the descent dynamics.*

The potential provides a scalar summary of the constraint satisfaction state of a history. It is analogous to the energy functional in statistical-mechanical lattice models, where the equilibrium configuration minimizes the system's energy under local interaction constraints. In computation, stable histories correspond to configurations in which no further local transition reduces the degree of constraint violation. Section 8 develops this correspondence in greater detail.

8 Time, Causality, and Irreversibility

The event-historical framework does not presuppose time as a primitive dimension; rather, temporal structure emerges from within the framework as a property of the ordering relation over histories. This emergence is not metaphorical but structural: the algebraic properties of the history poset directly induce a temporal asymmetry that mirrors the thermodynamic arrow of time.

8.1 Temporal Structure from Causal Order

The prefix ordering $\preceq$ on $\mathcal{H}$ induces a natural temporal direction. If $H_1 \prec H_2$, then H_2 is the later history: it contains all of H_1 's events together with additional events that post-date them. The

progression of execution corresponds to movement upward in the prefix ordering, and the execution time of a history may be measured by its depth in this ordering.

In partially ordered histories, time is not a single global parameter but a family of local parameters indexed by causal chains within the history. Lamport’s logical clocks [11] assign a numerical timestamp to each event in such a way that causally related events receive increasing timestamps, making this local temporal structure globally accessible.

8.2 Two Modes of Irreversibility

The framework distinguishes two structurally distinct forms of irreversibility that operate simultaneously and reinforce each other.

The first is *execution irreversibility*: histories grow monotonically under extension, and events once incorporated into a history cannot be removed without introducing compensating events that become part of the extended record. This irreversibility is formal: it follows directly from the monotonicity theorem (1) and the definition of the prefix order.

The second is *abstraction irreversibility*: reductions discard information about the causal ordering and intermediate structure of histories, and this discarded information cannot in general be recovered from the reduced representation alone. This irreversibility is information-theoretic: it follows from the non-injectivity of reduction operators established in Proposition 2.

These two forms of irreversibility interact in a manner that closely parallels the relationship between microscopic dynamics and macroscopic thermodynamics. At the microscopic level, transitions accumulate irreversibly, extending the causal structure of the history. At the macroscopic level, reductions compress histories into coarse-grained representations, discarding microscopic detail and increasing the number of histories compatible with any given reduced state. The entropy-like quantity

$$S(r) = \log \Omega(\rho, r) = \log |\{H \in \mathcal{H} : \rho(H) = r\}|$$

measures this macroscopic indeterminacy: it grows as reduction discards finer detail, in exact analogy with the Boltzmann entropy of a thermodynamic macrostate.

8.3 Event Histories and Constraint Propagation in Physical Systems

The structural correspondence between the event-historical kernel and physical systems governed by local constraint propagation can be made precise through the Ising model. In this model, a lattice of N sites carries spin variables $s_i \in \{-1, +1\}$, and the system’s energy is given by

$$E(\mathbf{s}) = -J \sum_{\langle i,j \rangle} s_i s_j - h \sum_i s_i,$$

where J is the spin-coupling constant, h is an external field, and $\langle i, j \rangle$ ranges over neighboring pairs.

Each spin flip at site i constitutes an event $e_k = (i_k, \Delta s_{i_k})$ in the sense of the kernel's event universe. The sequence of spin flips executed by the dynamics forms an event history

$$H = (e_1, e_2, \dots, e_n),$$

and the instantaneous spin configuration $\mathbf{s} = (s_1, \dots, s_N)$ is a state projection $\sigma(H) = \mathbf{s}$ derived from this history by replay. The energy $E(\mathbf{s})$ then defines a monotone potential in the sense of the history algebra: Metropolis or Glauber dynamics accept transitions that reduce energy, and the system descends toward stable configurations that minimize E subject to thermal fluctuations.

The constraint propagation structure is apparent in the local character of spin dynamics: a flip at site i alters the effective field experienced by i 's neighbors, reshaping the likelihood of their subsequent transitions. The causal ordering of events in the history therefore records not only the temporal sequence of flips but the propagation of interaction constraints through the lattice. The system's configuration at any moment is a reduction of the underlying spin-flip history, and the energy landscape over which descent occurs corresponds to the monotone potential of the history algebra.

This correspondence is not an analogy but a structural identity: Ising dynamics and constraint-driven event-historical computation are instances of the same abstract process—the monotonic extension of a causally ordered history under a locally propagated potential.

8.4 Monotone Constraint Dynamics

The general formulation of constraint propagation within the history algebra proceeds as follows. Let $\mathcal{C}(H)$ denote the constraint set induced by history H . The extension of H by an event e updates this constraint set according to a local update function F :

$$\mathcal{C}(H \cdot e) = F(\mathcal{C}(H), e).$$

In systems where constraints accumulate without removal—where the occurrence of an event either introduces new constraints or strengthens existing ones—the constraint set evolves monotonically:

Principle 6 (Constraint Monotonicity). *For histories $H_1 \preceq H_2$, the constraint sets satisfy $\mathcal{C}(H_1) \subseteq \mathcal{C}(H_2)$.*

Constraint monotonicity implies that execution does not remove previously established dependencies; it only introduces new ones. Combined with the monotone potential, this gives rise to

descent dynamics in which each local event reduces the potential by satisfying or propagating constraints, guiding the history toward stable configurations in which all local constraints are mutually satisfied.

9 Unified Constraint-Driven Computation

The minimal operational kernel is not a model of any particular computational paradigm. It is an abstract characterization of a structural pattern that appears across multiple domains. The preceding sections have developed the kernel’s algebraic structure; the present section traces its instantiation across a range of concrete systems, demonstrating that the event-historical framework captures a genuine organizational unity beneath apparent diversity.

9.1 Deterministic Replay Systems

Event-sourced architectures [7, 10] are perhaps the most direct computational instantiation of the kernel. In these systems, the primary persistent datum is not a mutable application state but an append-only log of events. Each log entry is an event in the kernel’s sense; the cumulative log is the event history H ; and the application state visible to users is the projection $\sigma(H)$ obtained by replaying the log. Checkpoints and snapshots are reduction operators that compress the history to accelerate state reconstruction, while preserving the log as the authoritative source. Failures are recovered by replaying the log from a checkpoint; this corresponds exactly to the causal correctness principle established in Section 4.

9.2 Distributed Convergence

Conflict-free replicated data types [17] achieve eventual consistency across distributed replicas without centralized coordination by ensuring that local update operations form a join-semilattice. Each replica maintains an operation history that grows monotonically as updates are applied. Synchronization between replicas proceeds through merge operations that combine their respective histories while preserving causal ordering. Because the merge operation is associative, commutative, and idempotent—the defining properties of a semilattice join, established for the general case in Proposition 2—repeated synchronization among any collection of replicas converges to a unique consistent merged history regardless of the order in which synchronizations occur. The consistency guarantee is therefore a structural consequence of the algebra of histories rather than a property imposed by a coordination protocol.

9.3 Version-Control Systems

Distributed version-control systems such as Git [4] realize the history algebra with particular clarity. A repository is organized as a directed acyclic graph of commits, where each commit is an event extending the history of the project. The commit graph is an explicit externalization of the prefix order over histories; branching corresponds to the divergence of compatible histories from a common prefix; and merge commits implement the join operation, constructing a new history that incorporates multiple branches while preserving their internal causal ordering. The repository therefore does not merely store code revisions; it is a concrete instantiation of the event-historical algebra.

9.4 Constraint-Based Programming

Constraint-based programming systems compute solutions by propagating constraints through networks of variables rather than by executing explicit sequences of state transitions. Within the event-historical framework, each propagation step is an event that extends the computation history; the constraint set $\mathcal{C}(H)$ evolves monotonically as propagation proceeds; and the solution—the configuration satisfying all constraints—corresponds to a stable history in the sense of Section 7. The computation is not a search through a space of states but a trajectory through the lattice of histories guided by the monotone potential defined by the constraint satisfaction degree.

9.5 Common Algebraic Structure

Despite their apparent diversity, the systems described above share the same abstract organization. In each, evolution proceeds through the monotonic extension of a causally ordered history. Composition arises through join-like merge operations that preserve causal ordering. Derived representations emerge through reductions that compress histories into purpose-specific projections. Stable configurations appear as fixed points of the descent dynamics induced by the monotone potential.

This shared structure suggests that the event-historical kernel does not describe a computational model in competition with others but rather a foundational algebraic framework of which existing computational paradigms are concrete instantiations. The organizational unity underlying distributed logs, version graphs, constraint solvers, and physical lattice dynamics is not coincidental; it reflects the common mathematical structure of the history algebra developed in the preceding sections.

10 Conclusion

This essay has proposed and developed a minimal operational kernel for computation in which event histories serve as the primitive objects of the theory and states emerge as derived reductions of those histories. The motivating observation is that the state-centered ontology dominant in theoretical computer science systematically obscures the temporal, causal, and irreversible character of computation—features that are not incidental to real systems but constitutive of their behavior. Reversing the foundational relationship between state and history, taking the latter as primary and deriving the former, yields a framework that is both more faithful to the operational character of computation and richer in its algebraic organization.

The kernel is governed by three operations. Extension appends events to histories and so realizes execution as the irreversible accumulation of causally ordered transitions. Merge reconciles compatible histories through a join-like operation that preserves causal ordering and realizes composition as historical reconciliation rather than structural containment. Reduction maps histories to compressed representations through controlled information loss and realizes abstraction as systematic compression rather than the generation of new conceptual entities. The interaction of these three operations suffices to generate the full space of computationally relevant structures.

The algebraic structure induced by these operations is a join-semilattice over a partially ordered space of histories, admitting a monotone potential whose descent characterizes execution dynamics. Three principal theorems govern the framework: the monotonicity of extension (Theorem 1), the convergence of merge to a least upper bound (Theorem 2), and the uniqueness of replay under deterministic event semantics (Theorem 3). These are not independent engineering desiderata; they are structural consequences of the history algebra itself.

The framework’s implications extend beyond a reformulation of programming semantics. The event-historical algebra appears, under appropriate interpretation, in event-sourced architectures, conflict-free replicated data types, distributed version-control systems, constraint-based programming systems, and physical lattice models governed by local interaction potentials. In each of these domains, system behavior emerges from the monotonic accumulation of locally constrained events, composition is realized through join-like reconciliation of divergent histories, and abstraction is achieved through reduction operators that compress execution traces into purpose-specific projections. The convergence of these structural patterns across domains as different as distributed systems, software development tools, and statistical mechanics suggests that the event-historical kernel captures a genuine organizing principle rather than a domain-specific model.

The deeper significance of this unification is philosophical as much as technical. It reframes computation not as the traversal of a pre-existing state space but as an act of construction: the progressive building of irreversible causal structures through locally constrained transitions. State

ceases to be the foundational object of the theory and becomes instead a derived convenience, a compressed view of the history from which it was produced. Abstraction ceases to be the introduction of higher-level entities and becomes instead the systematic discarding of distinctions that are irrelevant to a given purpose. And the temporal asymmetry of computation—the fact that histories grow forward and cannot be undone—ceases to be an assumption imported from physics and becomes instead a structural theorem, a consequence of the irreversible extension operator that defines what it means to execute.

References

- [1] Samson Abramsky. *Proofs as Processes*. Theoretical Computer Science, 135(1):5–9, 1994.
- [2] Christel Baier and Joost-Pieter Katoen. *Principles of Model Checking*. MIT Press, 2008.
- [3] Stephen Brush. *History of the Lenz-Ising Model*. Reviews of Modern Physics, 39(4):883–893, 1967.
- [4] Scott Chacon and Ben Straub. *Pro Git*. Apress, 2014.
- [5] Thomas Cover and Joy Thomas. *Elements of Information Theory*. Wiley, 1991.
- [6] Brian Davey and Hilary Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2002.
- [7] Martin Fowler. *Event Sourcing*. martinowler.com, 2005.
- [8] John Hopcroft, Rajeev Motwani, and Jeffrey Ullman. *Introduction to Automata Theory, Languages, and Computation*. Pearson, 2006.
- [9] Ernst Ising. *Contribution to the Theory of Ferromagnetism*. Zeitschrift für Physik, 31:253–258, 1925.
- [10] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [11] Leslie Lamport. *Time, Clocks, and the Ordering of Events in a Distributed System*. Communications of the ACM, 21(7):558–565, 1978.
- [12] Ming Li and Paul Vitányi. *An Introduction to Kolmogorov Complexity and Its Applications*. Springer, 2008.
- [13] David MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.
- [14] Marc Mézard, Giorgio Parisi, and Miguel Virasoro. *Spin Glass Theory and Beyond*. World Scientific, 1987.
- [15] Robin Milner. *Communication and Concurrency*. Prentice Hall, 1989.
- [16] Mogens Nielsen, Gordon Plotkin, and Glynn Winskel. *Petri Nets, Event Structures and Domains*. Theoretical Computer Science, 13(1):85–108, 1995.

- [17] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. *Conflict-Free Replicated Data Types*. Stabilization, Safety, and Security of Distributed Systems (SSS), 2011.
- [18] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 2013.
- [19] Glynn Winskel and Mogens Nielsen. *Models for Concurrency*. Handbook of Logic in Computer Science, Oxford University Press, 1993.